

Robust Queries, Semantic Search & Geospatial Retrieval with Firestore Pipelines

Hassan Bahati

GDE, Firebase



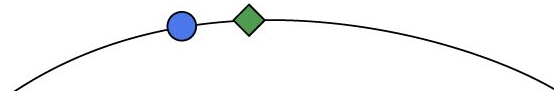
Firestore ?



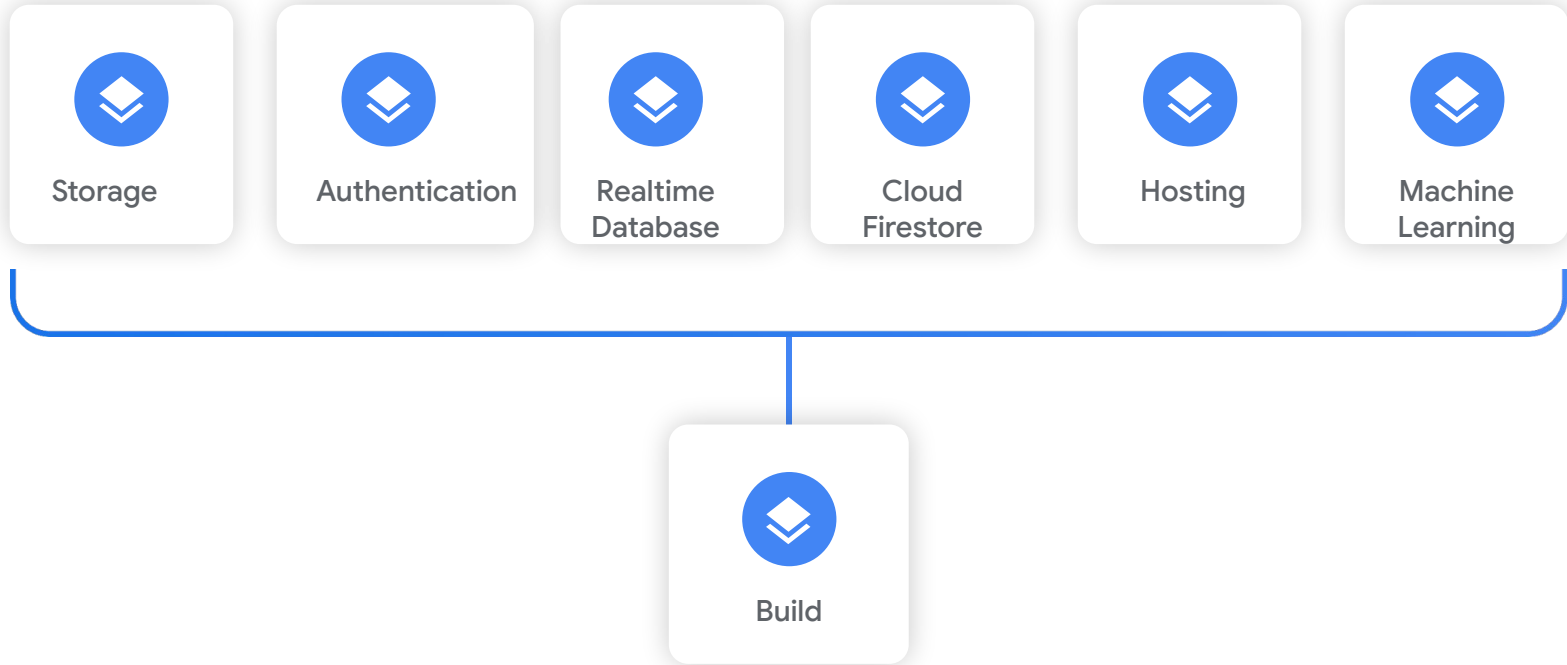
App development platform that helps developers build, launch, and manage mobile and web applications with ease.



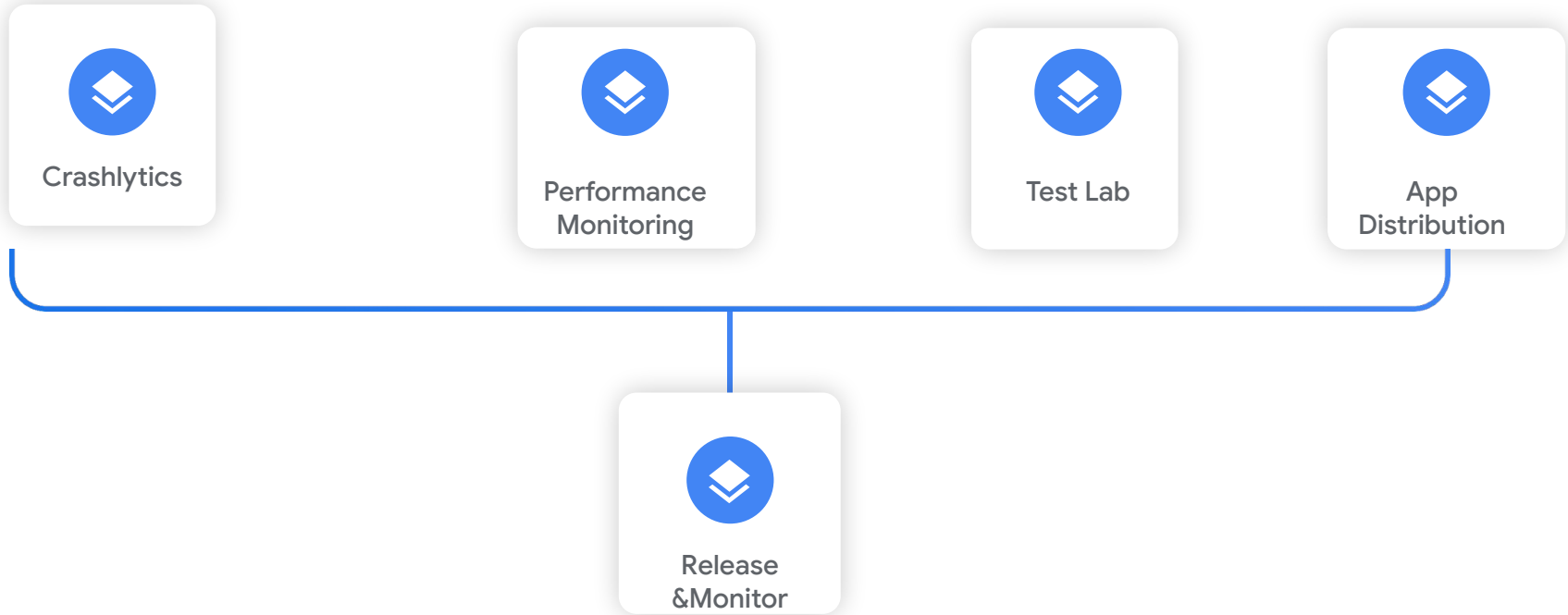
Firestore is a collection of tools and services that empower developers to succeed.



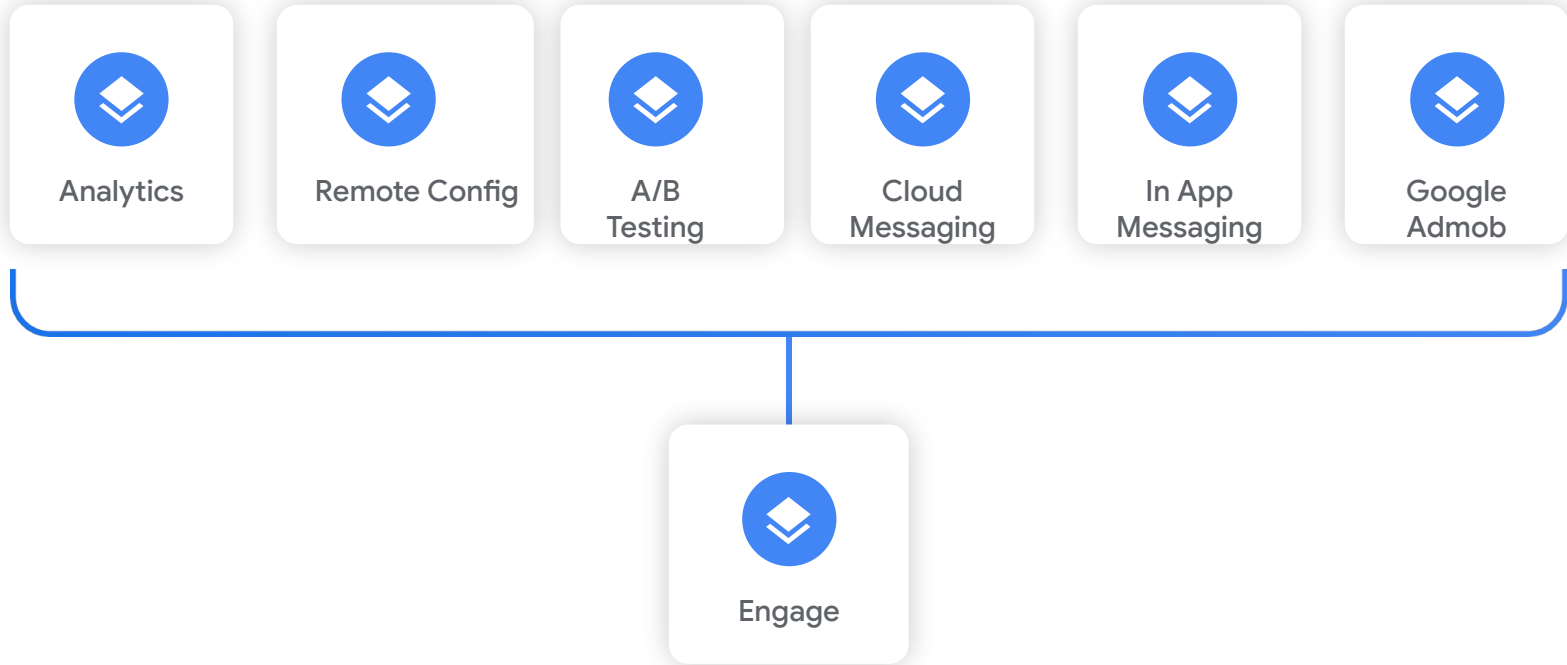
Build with Firebase



Release & Monitor with Firebase



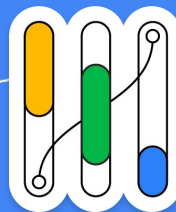
Engage with Firebase



Most apps eventually needs to answer this question:

"Find me something specific, near where I am."

How can I make my app faster ??

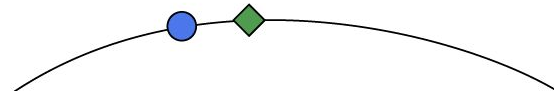


"Find me something specific, near where I am."

Coffee shops with WiFi.

Events happening this weekend within 5km.

Drivers available in your neighbourhood. It sounds like a simple feature request.



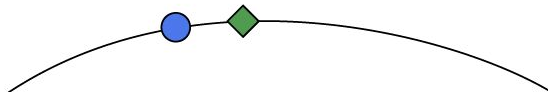
Prior you would end up with something like this

// Firestore as your source of truth

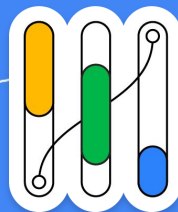
// Algolia or Typesense for text search

// A Cloud Function to sync data between them

// Geohash libraries and bounding box math for proximity



Three moving parts to
answer one user query.



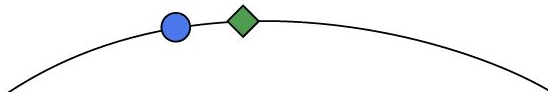
The Problem

We are trying to answer queries of this shape:



Given a collection D ,

return documents matching text T within radius R of point P – ideally ranked by relevance.

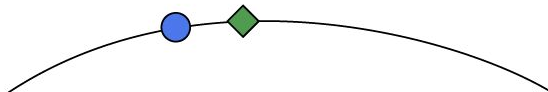


Three distinct problems:

// does this document match what the user typed?

// is this document close to the user?

// of everything that matches both, which result comes first?

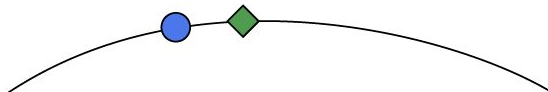


Why This Was Hard Before

// **Cloud Firestore** great at querying by exact field values. no concept of text relevance or distance.

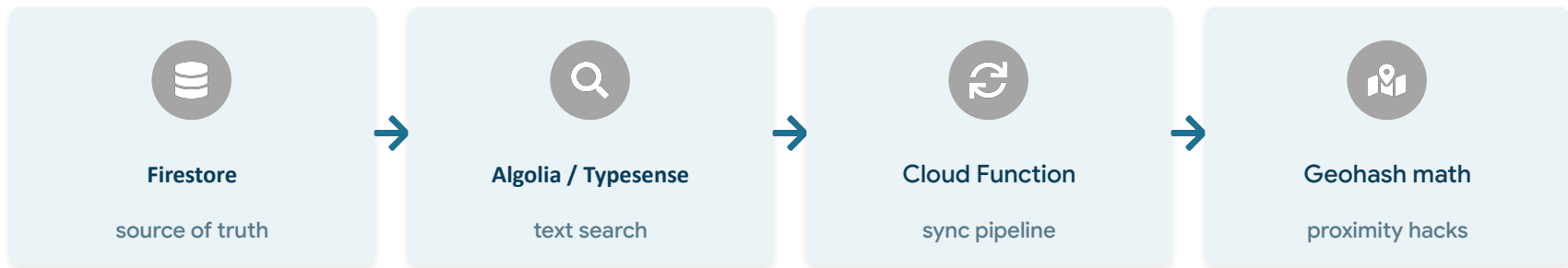
// **Algolia** excellent text relevance & good developer experience. It also had geo search. 3rd Party.

// **Elasticsearch** solved both text and geo natively and at scale. It also required you to run and tune a cluster, which is a significant operational commitment for a team that just wants to ship a feature.



“Find me something specific, near where I am.”

*Coffee shops with WiFi. Drivers nearby. Events within 5km. A simple
feature request — and historically, never simple to build on Firebase.*



What changed at I/O 2026

Firestore Enterprise Edition — Native mode with Pipeline operations

A stage-based,
composable query engine
— not just
method-chaining.

`documentMatches()` —
handles synonyms &
spelling errors server-side.

`geoDistance` & `findNearest`
stages, no more geohash
hacks.

What actually changes under the hood

Aspect	Standard Edition	Enterprise Edition (Native mode)
Indexing	Single-field indexes created automatically	Indexes are optional — you choose what to index
Unindexed query	Not really possible — indexes are automatic	Falls back to a full collection scan
Billing basis	Per document read / write / delete	Read Units (4KB) and Write Units (1KB) tranches
Query interface	Method-chained: <code>.where().orderBy()</code>	+ Pipeline operations: composable stages
Search & geo	Manual geohashes, external search service	Native: full-text search + geospatial stages

A query is a sequence of stages

Method-chaining gives you `.where()` and `.orderBy()`. Pipelines give you a composable chain of stages — each one transforming the output of the last.



Each stage receives the previous stage's output — this is what makes server-side joins, aggregation, search, and geo composable in one query.

From document reads to byte tranches



Standard Edition

Charged per document read, write, or delete — flat and predictable per operation.

1 read = 1 document

Simple to reason about. Cost scales with result-set size, not dataset size — as long as the query is indexed.



Enterprise Edition

Charged in Read Unit (4KB) and Write Unit (1KB) tranches — bytes, not document counts.

1 read = bytes ÷ 4KB

An unindexed query scans the full dataset — cost scales with bytes scanned, not just what's returned.

Automatic indexing vs. opt-in indexing



Standard Edition — Automatic

- Single-field indexes created for every field, automatically
- Composite indexes still need manual configuration
- More storage overhead, less to think about up front



Enterprise Edition — Opt-in

- No automatic indexing — you decide what's worth indexing
- Missing an index means a full collection scan, not an error
- Lower storage & write cost, but the responsibility shifts to you

Use Query Explain and Query Insights to see exactly which indexes would help, before you commit to one.

Full-text search, without leaving Firestore

The `search()` stage and `documentMatches()` expression bring relevance-ranked text search directly into a Pipeline query — no Algolia, no sync function.

Handles typos & synonyms

Built-in, no extra configuration

Relevance scoring via `score()`

Sort by how well a document matches

Runs server-side, in the pipeline

Combine with filters in one round trip

```
const results = await db.pipeline()
  .collection("places")
  .search(
    Field("description").documentMatches(
      "quiet space good wifi"
    )
  )
  .sort(Field("score").descending())
  .limit(10)
  .execute();
```

Geospatial retrieval, the same way



`geoDistance()` and the `findNearest()` stage replace geohash bucketing entirely — true radius search, computed server-side.

No more geohash bucketing

True distance, not square approximations

`findNearest()` sorts by proximity

Returns closest matches directly

Composable with search & filters

One round trip, not three services

```
const center = GeoPoint(0.3136, 32.5811);

const results = await db.pipeline()
  .collection("places")
  .findNearest(
    Field("location"),
    center,
    { limit: 10, radiusMeters: 3000 }
  )
  .execute();
```

What we're building today

A “near me” search feature for a places app — combining native text search and geospatial retrieval in a single Firestore Pipeline.



1. Data model

places collection with GeoPoint +
text fields



2. Search stage

`documentMatches()` for
relevance-ranked text



3. Geo stage

`findNearest()` for true radius
search



4. Combined pipeline

Search + geo + sort, in one query

Designing the places collection



`geoDistance()` and the `findNearest()` stage replace geohash bucketing entirely — true radius search, computed server-side.

Use `GeoPoint`, not separate `lat/lng` fields — it's a native type Pipeline geo stages expect directly.

Keep text fields intentional. `name` + `description` give `documentMatches()` real signal to rank on.

No geohash field needed. `findNearest()` computes true distance — that was a Standard-edition workaround.

```
{
  name: "Java House Kilimani",
  description: "Quiet space, fast wifi,
               great for remote work",
  tags: ["coffee", "wifi",
         "remote-work"],
  location: GeoPoint(-1.2921, 36.7820)
}
```

Why the data model?

// Use GeoPoint, not separate lat/lng fields.

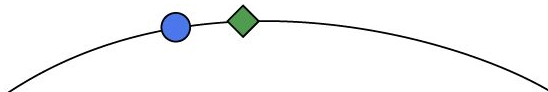
Firestore's GeoPoint type stores coordinates as an atomic unit. It works cleanly with the geo libraries you will use later and signals intent clearly in your schema

// Precompute the geohash at write time.

This is a string like "k9f3m7" that encodes a geographic region. You will need it for Strategy 3. Generate it using geofire-common when you create or update a document — do not try to compute it at query time.

// Keep text fields intentional.

The more text fields you add, the broader your search surface. Be deliberate about what you are matching against — name, description, and tags cover most use cases without over-indexing.



A first Pipeline query

```
const db = getFirestore(app);

const results = await db.pipeline()
  .collection("places")
  .where(
    Field("tags").arrayContains("wifi")
  )
  .limit(20)
  .execute();

results.forEach(doc =>
  console.log(doc.data()));
```

What's different from Core operations:

- `db.pipeline()` enters Pipeline mode explicitly
- `Field(...)` wraps each field reference
- Stages chain in sequence: `collection` → `where` → `limit`
- Unindexed? It still runs — just scans more bytes

Layering in relevance-ranked search

```
const userQuery = "quiet wifi remote  
work";  
  
const results = await db.pipeline()  
  .collection("places")  
  .search(  
    Field("description")  
      .documentMatches(userQuery)  
  )  
  .sort(Field("score").descending())  
  .limit(10)  
  .execute();
```

Free natural-language input

`documentMatches()` understands free text — typos and synonyms included — and `score()` gives you a real relevance signal to sort by, something array-contains could never do.

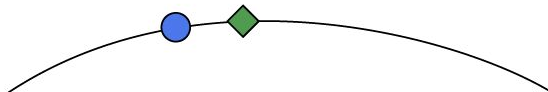
Layering in true proximity search

```
const userLocation = GeoPoint(-1.2864,  
36.8172);  
  
const results = await db.pipeline()  
  .collection("places")  
  .findNearest(  
    Field("location"),  
    userLocation,  
    { limit: 15, radiusMeters: 3000 }  
  )  
  .execute();
```

True distance, not approximated

`findNearest()` computes real geodesic distance and sorts by it directly — no rectangular geohash bounds, no edge-of-radius false positives to filter out afterward.

Putting It Together: A Complete Example

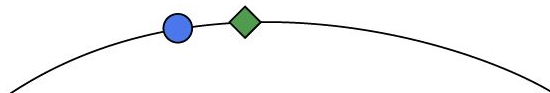


Search + geo, in one server-side pipeline

- ✓ One query. One round trip.
- ✓ No Algolia. No sync function. No geohash math.
- ✓ Relevance and proximity, ranked together.

```
const results = await db.pipeline()  
  .collection("places")  
  .search(  
    Field("description").documentMatches(  
      "quiet wifi remote work"  
    )  
  )  
  .findNearest(  
    Field("location"),  
    GeoPoint(-1.2864, 36.8172),  
    { radiusMeters: 3000 }  
  )  
  .sort(Field("score").descending())  
  .limit(10)  
  .execute();
```

Helper to write a place with the geohash

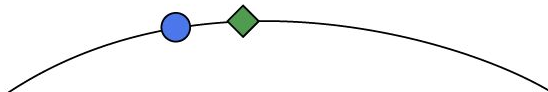




```
import { geohashForLocation } from "geofire-common";
import { collection, addDoc } from "firebase/firestore";
import { GeoPoint } from "firebase/firestore";

async function addPlace(place) {
  const geohash = geohashForLocation([
    place.lat,
    place.lng
  ]);

  await addDoc(collection(db, "places"), {
    name: place.name,
    description: place.description,
    tags: place.tags,
    location: new GeoPoint(place.lat, place.lng),
    geohash
  });
}
```

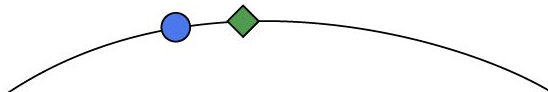


Ranking

All three strategies above produce a filtered list. None of them produce a ranked list.

A user expects:

“The most relevant result near me should come first — not just any result within 3km.”



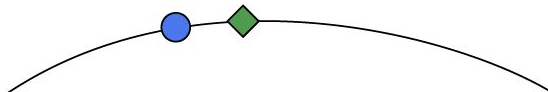
The scoring function



$$\text{score}(d) = \alpha \times \text{text_relevance}(d) + \beta \times (1 / \text{distance}(d, P))$$

Where α and β are weights you tune based on how much text relevance vs. proximity should influence the result order.

Firestore does not support this server-side. The ranking has to happen in your application layer, after you have the candidates.





```
function rankResults(results, userLocation) {  
  return results  
    .map(place => {  
      const distance = distanceBetween(  
        [place.location.latitude, place.location.longitude],  
        userLocation  
      );  
      return { ...place, distance };  
    })  
    .sort((a, b) => a.distance - b.distance);  
}
```

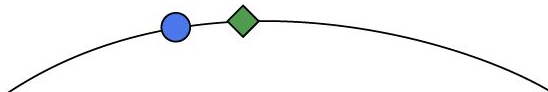


What to watch before you ship this

// **Unindexed scans cost real money:** An unindexed Pipeline query scans the full collection in bytes — Read Units accumulate fast on large datasets.

// **Profile before you commit** Use Query Explain to see Read/Write Unit consumption per query, and Query Insights to find what's worth indexing.

// **New index types, new tradeoffs** Sparse, non-sparse, and unique indexes give finer control — but each one is a write-cost decision now, not a default.



When is this actually worth adopting?



Reach for Pipelines if...

- You need text + geo search in a single query
- You're paying for Algolia/Elasticsearch just for basic relevance
- Your team can own index decisions deliberately
- Preview status is acceptable for your timeline



Stay with Standard if...

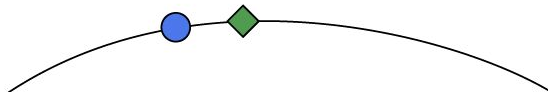
- Your queries are simple, exact-match lookups
- You need GA stability guarantees right now
- Automatic indexing is genuinely saving you effort
- Your dataset is small enough that cost isn't a factor yet

What to carry out of this room

// Pipeline operations are a different query model — stage-based, composable, worth learning on its own terms.

// The billing model shift (Read/Write Units) is the single most important thing to understand before migrating.

// Native search + geo collapse a multi-service architecture into one database, for a real class of apps.



Resources

1. firebase.google.com/docs/firestore/enterprise/pipelines-overview
2. firebase.google.com/docs/firestore/pipelines/get-started-with-pipelines
3. firebase.google.com/docs/firestore/enterprise/pricing-examples
4. firebase.google.com/docs/firestore/enterprise/text-search
- 5.

